

OOP Sessional Exam: Past Paper Analysis

Nirma University, MCA Semester I | Java OOP | Based on 4 final-exam papers (Feb-2022, Jan-2023, Jan-2025, Dec-2025) mapped to your sessional syllabus

KEY FINDINGS

- **Scope matches your estimate:** on average about 57% of final-exam marks fall on your sessional topics (range 44% to 70% per paper).
- **Three units appeared in all 4 papers:** JDK/JRE/JVM, Loops, and Constructors. Constructor vs Method was asked in every single paper, usually for 8 marks.
- **Top 5 to prioritize:** (1) Constructor vs Method, (2) JDK / JRE / JVM, (3) Loops, (4) String class methods, (5) Static variable and static method. Together these made up about 67% of all sessional-topic marks (126 of 187).
- **Heads-up:** String.replace() and String.contains() were asked in past papers but are not in your slide text, so plan some self-study for those two.

0. Scope Check: How Much of Each Paper Is Sessional Content?

Paper	Format	Marks on sessional topics	% of paper	Questions outside your sessional (ignored in this analysis)
Feb-2022 (3MCA101)	50 marks, 2 hrs	35 / 50	70%	Q1C JDBC / streams (OR options), Q3A exceptions, Q3B-OR abstract class and interface
Jan-2023 (6CS101)	100 marks, 3 hrs	64 / 100	64%	JDBC, overloading vs overriding, interface, exceptions, I/O streams (Q4B, Q4C, Q5B, Q6A, Q6B)
Jan-2025 (6CS101CC22)	100 marks, 3 hrs	38 to 44 / 100	38 to 44%	JDBC, threads, packages, interface, polymorphism, access modifiers, file I/O, exceptions. Q1A (OOP features) is borderline, hence the range
Dec-2025 (6CS101CC25)	100 marks, 3 hrs	50 / 100	50%	JDBC, packages, threads, abstract class / interface, exceptions, overloading vs overriding, I/O streams

1. Topic Weightage Table

Frequency = number of the 4 past papers in which the topic appeared. Marks are the marks assigned to that sub-question in the final exam. Units follow the slide-deck headings of your sessional syllabus (the file has no formal unit labels).

1.1 Unit roll-up (where the marks actually are)

Unit (slide numbers)	Papers with 1+ question	Marks across 4 papers	Share of sessional marks	Priority
U8 · Constructors (slides 158 to 172)	4/4	31	16.6%	HIGH
U1 · Java Fundamentals: JDK / JRE / JVM (1 to 11)	4/4	26	13.9%	HIGH
U9 · Static variable and static method (173 to 181)	3/4	24	12.8%	HIGH
U5 · Loops and Arrays (104 to 120)	4/4	23	12.3%	HIGH
U6 · String Handling (121 to 138)	3/4	22	11.8%	HIGH
U3 · Decision Making: if / else-if / switch / break / continue (73 to 89)	2/4	16	8.6%	MEDIUM
U4 · Operators and Type Conversion (90 to 103)	3/4	16	8.6%	MEDIUM
U10 · Inheritance (182 to 191)	2/4	15	8.0%	MEDIUM
U7 · Methods and Classes (139 to 157)	1/4	8	4.3%	LOW
U2 · Tokens, Variables, Data Types (11 to 25)	1/4	6	3.2%	LOW

Total sessional-topic marks counted: 187 (excludes the 6-mark Jan-25 OOP-features question, which is borderline). Slide numbers refer to the first copy of the deck; slides 1 to 36 repeat as 37 to 72.

1.2 Specific topics (every question mapped)

Unit	Specific topic	Frequency (of 4)	Appeared in (paper, question no.)	Typical marks	Priority
U1 · Java Fundamentals	JDK vs JRE vs JVM : role of JDK / JVM, JDK components, "write once, run anywhere"	4/4	Feb-22 Q1A · Jan-23 Q1A · Jan-25 Q2A · Dec-25 Q1B	6 to 8 (usually 6)	HIGH
U1 · Java Fundamentals	Features of Java / features of OOP	1/4	Jan-25 Q1A (borderline scope)	6	LOW
U1 · Java Fundamentals	Java PATH setup, compile and run steps, Eclipse	0/4	Not asked	-	LOW
U2 · Tokens, Variables, Types	Java tokens: keywords, identifiers, literals, operators, separators	1/4	Jan-23 Q1B	6	MEDIUM

Unit	Specific topic	Frequency (of 4)	Appeared in (paper, question no.)	Typical marks	Priority
U2 · Tokens, Variables, Types	Variables, primitive and non-primitive data types	0/4	Not asked	-	LOW
U3 · Decision Making	else-if ladder (vs simple if-else) with real-life program	1/4	Dec-25 Q2B	8	MEDIUM
U3 · Decision Making	Switch statement with example	1/4	Jan-23 Q1C	6	MEDIUM
U3 · Decision Making	break / continue (as code-output tracing)	1/4	Dec-25 Q1A(2)	2	LOW
U4 · Operators	Relational operators + example (and logical operators in code tracing)	3/4	Feb-22 Q1B · Jan-23 Q2A · Dec-25 Q1A(3) (as code snippet)	6 to 8 (2 as snippet)	MEDIUM
U4 · Operators	Arithmetic, bitwise, assignment operators, precedence, type conversion	0/4	Not asked	-	LOW
U5 · Loops and Arrays	Types of loops: importance, while vs do-while	2/4	Feb-22 Q2B · Jan-25 Q1B	6 to 7	HIGH
U5 · Loops and Arrays	Nested loops: pattern-printing program	1/4	Jan-23 Q3A	8	HIGH
U5 · Loops and Arrays	For-loop output tracing	1/4	Dec-25 Q1A(1)	2	HIGH
U5 · Loops and Arrays	Arrays (1-D, declaration, length)	0/4	Not asked	-	LOW
U6 · String Handling	String class methods: substring() in all three; plus equals, concat, compareTo, contains, replace, length	3/4	Jan-23 Q2B · Jan-25 Q3A · Dec-25 Q1C	6 to 8	HIGH
U6 · String Handling	toUpperCase, toLowerCase, trim, charAt	0/4	Not asked	-	LOW
U7 · Methods and Classes	Write a class with fields and methods (Employee, gross salary = salary + HRA + DA)	1/4	Dec-25 Q3A	8	MEDIUM
U7 · Methods and Classes	Parameter passing (by value / by reference), return types	0/4	Not asked	-	LOW
U8 · Constructors	Constructor vs method: name, return type, purpose, invocation + program	4/4	Feb-22 Q2A · Jan-23 Q3B · Jan-25 Q3B · Dec-25 Q3B	7 to 8 (usually 8)	HIGH
U9 · Static	Static method vs non-static (accessibility, usage) + program	2/4	Dec-25 Q2A · Jan-23 Q5A (static var and method relationship)	8	HIGH
U9 · Static	Static vs non-static variables with example	2/4	Jan-25 Q2B · Jan-23 Q5A	8	HIGH

Unit	Specific topic	Frequency (of 4)	Appeared in (paper, question no.)	Typical marks	Priority
U10 · Inheritance	Types of inheritance: multilevel (program), hierarchical (example)	2/4	Feb-22 Q3B · Jan-23 Q4A	6 to 9	MEDIUM
U10 · Inheritance	Single and multiple inheritance, advantages of inheritance	0/4	Not asked	-	LOW
Cross-unit	Code output / error tracing with justification (3 snippets)	1/4	Dec-25 Q1A	6 (2 each)	MEDIUM

2. High-Yield Priority List

Study these five in this order. They are the five highest-mark units in the roll-up above, and each has appeared in at least 3 of 4 papers.

Rank	Topic	Evidence from past papers	What to prepare for full marks
1	Constructor vs Method	Asked in 4/4 papers , 7 to 8 marks each time (31 marks in total, the biggest single block). Feb-22 and Jan-23 use almost identical wording. Dec-25 adds "program using both a constructor and a method".	- Definition of a constructor, plus the default constructor (supplied by the compiler only when you write none). - Comparison table: name (same as class vs any name), return type (none, not even void), purpose (initialize object vs perform behaviour), invocation (automatic on new vs explicit call), calls per object. - One program with a parameterized constructor, a default constructor and a method, with output (model it on the Customer / Circle slides). - Mention constructor overloading in one line.
2	JDK vs JRE vs JVM	Asked in 4/4 papers , 6 to 8 marks. Variants: distinguish all three (Jan-23), role of JDK (Feb-22), JDK components (Jan-25), importance of JVM and "write once, run anywhere" (Dec-25).	- JDK = development tools (javac, java) + JRE; JRE = JVM + libraries; JVM = runtime that executes the program. - Flow diagram: .java source, javac, .class bytecode, JVM, machine code on any OS. - A 3-column comparison table plus a 2-line "why JVM gives write once, run anywhere" answer. - Note: the words "bytecode" and "write once, run anywhere" are not in the extractable slide text, so add them from your notes or textbook.
3	Loops	Appeared in 4/4 papers : types of loops (Feb-22), nested-loop pattern (Jan-23), while vs do-while (Jan-25), for-loop output tracing (Dec-25).	- Syntax and flowchart for while, do-while, for and enhanced for. - Entry-controlled (while, for) vs exit-controlled (do-while, runs at least once). - Pattern programs with nested loops. For the Jan-23 pattern (1 / 0 1 / 1 0 1 ...): inner loop j = 1..i, print 1 if (i + j) is even, else 0. - Practise dry-running loops with unusual steps (e.g. i = i + 11).

Rank	Topic	Evidence from past papers	What to prepare for full marks
4	String class methods	Asked in 3/4 papers, including each of the last three. Always three named methods; substring() appeared every time .	- For each method: signature, return type and a one-line example with output: substring(begin) and substring(begin, end) (end is exclusive), equals, concat, compareTo (0 / negative / positive), length. - equals() vs ==, and equals vs equalsIgnoreCase. Self-study: contains() and replace() were asked (Jan-25, Dec-25) but are not in your slide text.
5	Static variable and static method	Asked in 3/4 papers (Jan-23, Jan-25, Dec-25), 8 marks every time . A static question has appeared in every paper since 2023.	- Static vs instance variable: one shared copy per class vs one per object; access via the class name. - Static vs non-static method: called without an object; can access only static members directly; cannot use this (slide 181); non-static methods can access both. - Static block for initialization. - A program using the slides' Customer.deliveryCharge example with a static getter and setter.

Next tier (cheap marks if you have time left)

Topic	Frequency	Quick-win checklist
Relational and logical operators	3/4	Table of ==, !=, >, <, >=, <= (all return boolean) with a short program; && / / ! and how compound conditions evaluate (needed for output tracing). About 30 minutes of work.
Decision making: else-if ladder, switch	2/4	else-if ladder vs simple if-else, with a real-life program (grade or discount calculator). Switch: syntax, break, default, example.
Inheritance types	2/4	Single, multilevel, hierarchical, multiple (not supported in Java for classes) and advantages. One extends-based program (Customer, RegularCustomer) and one multilevel chain.
Tokens	1/4	Keywords, identifiers (naming rules), literals (integer, floating point, character, string, boolean), operators, separators.

3. Question Patterns

#	Recurring pattern	Evidence	How to answer
1	"Differentiate / Distinguish X and Y"	9 of the 27 sessional-topic questions are comparisons: constructor vs method (4/4), JDK vs JRE vs JVM, while vs do-while, static vs non-static, else-if ladder vs if-else, replace / length / substring.	Lead with a comparison table (4 to 6 rows), then add one short example.
2	"...with a suitable example / program"	Nearly every 6 to 9 mark theory question is paired with code (static, constructor, inheritance, else-if, loops).	Definition, then key points, then a compact program with its output.

#	Recurring pattern	Evidence	How to answer
3	Questions recycled almost word-for-word	Feb-22 Q2A = Jan-23 Q3B (constructor vs method, identical wording). Feb-22 Q1B = Jan-23 Q2A (relational operators). Constructor vs method then returns in Jan-25 Q3B and Dec-25 Q3B.	Past papers are the best predictor here. Prepare ready-made answers to these exact prompts.
4	String methods: a named trio, one always substring()	Jan-23: equals / concat / substring. Jan-25: compareTo / contains / substring. Dec-25: replace / length / substring.	Learn about 8 to 10 String methods, each with an example and output. The trio changes, the question does not.
5	Write a class from a real-life scenario	Employee with gross salary (Dec-25), real-life else-if (Dec-25), multilevel inheritance (Feb-22), hierarchical inheritance (Jan-23), static demo (Jan-23).	Keep a few reusable domains ready (Customer, Employee, Student, Bank) and adapt them. Your slides already use Customer and Student.
6	Output tracing and pattern printing	Dec-25 Q1A: three snippets worth 2 marks each (for-loop with step 11, continue, compound && conditions). Jan-23 Q3A: alternating 1/0 triangle with nested loops.	Dry-run line by line and state why (justification is explicitly requested). See the worked answers below.
7	Your sessional topics map to CO1 and CO2 only	Every sessional-topic question is tagged CO1 or CO2 (BL2 to BL6). CO3 is JDBC, threads, packages, I/O and exceptions, none of which you are tested on.	Expect "explain / differentiate / write a program" rather than recall-only questions.
8	Section I is the sessional section	Section I of the 100-mark papers was 50/50 sessional topics in Dec-25 and Jan-23, and 44/50 in Jan-25.	If your sessional mirrors Section I, expect JDK / JVM, one operators-or-control-flow item, a String question and a constructor question.

Worked answers: Dec-2025 Q1A (code-output pattern)

Snippet	Output	Justification
(1) for (int i = 55; i <= 100; i = i + 11)	For Loop: i = 55 i = 66 i = 77 i = 88 i = 99	i starts at 55 and grows by 11 each pass. After 99 the next value is 110, which fails i <= 100, so the loop stops.
(2) for 1 to 10 with: if (i == 7) continue;	1 2 3 4 5 6 8 9 10 (each on its own line)	continue skips the rest of the loop body when i is 7, so 7 is never printed, and the loop moves on to i = 8.
(3) x = 200, y = 50, z = 100 with three if blocks	Java	Condition 1: x > y is true, but y > z (50 > 100) is false, so it is skipped. Condition 2: z > y (100 > 50) and z < x (100 < 200) are both true, so "Java" prints. Condition 3: y + 200 = 250 > x is true, but y + 150 = 200 < z (100) is false, so it is skipped.

4. Gaps and Caveats

Topic	Note
Syllabus topics never asked (0/4)	Variables and data types, arrays, arithmetic / bitwise operators and type conversion, method parameter passing, toUpperCase / trim / charAt, single and multiple inheritance, Java PATH setup. They did not appear in the finals but are exactly the kind of content a shorter sessional could test as MCQs or short answers, so skim them last.
Asked but not in your slide text	String.contains(), String.replace(), "bytecode" and "write once, run anywhere" wording for JVM. A few slides are image-only, so check them and your textbook.
Format caveat	These are final-exam papers (50 or 100 marks). Your sessional may be shorter or use different formats, so treat the frequencies as topic likelihood, not a guarantee of question style.
Borderline item	Jan-25 Q1A (features of OOP) is only partly covered by your slides (features of Java, inheritance), so it was counted separately and excluded from the 187-mark total.